

Nome: Cauê, Guilherme, Luiz Felipe, Gabriel Toledo

Orientador: Hudson Neves

TCHESCO BARBERSHOP — Documentação Completa do Projeto

Índice

1. [Visão Geral](visão-geral)
2. [Tecnologias Utilizadas](tecnologias-utilizadas)
3. [Estrutura de Pastas](estrutura-de-pastas)
4. [Instalação e Execução](instalação-e-execução)
5. [Variáveis de Ambiente](variáveis-de-ambiente)
6. [API – Endpoints do Backend](api--endpoints-do-backend)
7. [Banco de Dados (In-Memory)](banco-de-dados-in-memory)
8. [Frontend – Páginas e Componentes](frontend--páginas-e-componentes)
9. [Autenticação e Segurança](autenticação-e-segurança)
10. [Testes](testes)
11. [Scripts Disponíveis](scripts-disponíveis)
12. [Fluxo da Aplicação](fluxo-da-aplicação)

Visão Geral

O Tchesco Barbershop é uma aplicação web completa (full-stack) desenvolvida para gerenciar os serviços de uma barbearia premium, metodologia ágil. O sistema permite que clientes:

- Visualizem os serviços disponíveis (cortes, barba, combos)
- Criem uma conta e façam login
- Agendem horários online
- Acessem a loja de produtos

Administradores têm acesso a um ****painel de controle (Dashboard)**** para visualizar todos os agendamentos.

Tecnologias Utilizadas

Backend

Tecnologia	Versão	Função
Node.js	—	Runtime JavaScript
Express	^5.2.1	Framework HTTP / servidor de API
JSON Web Token	^9.0.3	Autenticação via tokens JWT
bcryptjs	^3.0.3	Hash seguro de senhas
dotenv	^17.3.1	Gerenciamento de variáveis de ambiente
cors	^2.8.6	Controle de acesso entre origens
nodemon	^3.1.14	Reinício automático em desenvolvimento

Frontend

Tecnologia	Versão	Função
React	^19.2.4	Biblioteca de interface
Vite	^8.0.1	Bundler / servidor de desenvolvimento
React Router DOM	^7.13.1	Roteamento de páginas (SPA)
Axios	^1.13.6	Requisições HTTP para a API
Lucide React	^0.577.0	Ícones modernos
Vitest	^4.1.6	Framework de testes unitários
Testing Library	^16.3.2	Testes de componentes React

Estrutura de Pastas

Barbearia

```
|— backend/
|  |— config/
|  |  |— db.js          # Banco de dados in-memory + seed inicial
|  |— controllers/
|  |  |— authController.js  # Registro e login de usuários
|  |  |— appointmentController.js # Criação e listagem de agendamentos
|  |  |— serviceController.js # Listagem e criação de serviços
|  |  |— productController.js # Listagem de produtos e pedidos
|  |— middleware/
|  |  |— authMiddleware.js  # Verificação JWT e permissão de admin
|  |— routes/
|  |  |— authRoutes.js      # Rotas: /api/auth
|  |  |— appointmentRoutes.js # Rotas: /api/appointments
|  |  |— serviceRoutes.js    # Rotas: /api/services
|  |  |— productRoutes.js    # Rotas: /api/products
|  |— server.js             # Ponto de entrada do servidor
|  |— seed.js               # Script de seed auxiliar
|  |— test_api.js           # Script de testes manuais de API
|  |— .env                  # Variáveis de ambiente (não versionar)
|  |— package.json
|
```

```
└─ frontend/
    └─ public/          # Arquivos estáticos (imagens, ícones)
    └─ src/
        └─ components/
            └─ Navbar.jsx    # Barra de navegação principal
            └─ Icons.jsx     # Componentes de ícones SVG
            └─ pages/
                └─ Home.jsx   # Página inicial (hero, serviços, diferenciais)
                └─ Login.jsx  # Autenticação do usuário
                └─ Register.jsx # Cadastro de novo usuário
                └─ Scheduling.jsx # Agendamento de serviços
                └─ Store.jsx   # Loja de produtos
                └─ Dashboard.jsx # Painel administrativo
            └─ tests/
                └─ Autenticacao.test.jsx    # Testes de autenticação
                └─ InterfaceDoUsuario.test.jsx # Testes de UI
                └─ ProcessosDeNegocio.test.jsx # Testes de regras de negócio
            └─ App.jsx          # Componente raiz com rotas
            └─ main.jsx         # Ponto de entrada React
            └─ index.css        # Estilos globais e design system
            └─ App.css          # Estilos do App
└─ package.json
```

Instalação e Execução

Pré-requisitos

- [Node.js](https://nodejs.org/) versão 18 ou superior instalado.
- Terminal (PowerShell, CMD ou similar).

1. Clonar / navegar até o projeto

```
``bash  
  
cd Desktop/barbearia  
``
```

2. Instalar dependências do Backend

```
``bash  
  
cd backend  
  
npm install  
``
```

3. Instalar dependências do Frontend

```
``bash  
  
cd ../frontend  
  
npm install  
``
```

4. Iniciar o Backend (em um terminal separado)

```
``bash
```

```
cd backend
```

```
npm run dev    # modo desenvolvimento com nodemon
```

```
# ou
```

```
npm start      # modo produção
```

```
``
```

O servidor iniciará em: **http://localhost:5000**

5. Iniciar o Frontend (em outro terminal)

```
``bash
```

```
cd frontend
```

```
npm run dev
```

```
``
```

A aplicação abrirá em: **http://localhost:5173**

Variáveis de Ambiente

Crie o arquivo `backend/.env` com o seguinte conteúdo:

```
``env
```

```
JWT_SECRET=sua_chave_secreta_aqui
```

```
PORT=5000
```

```
``
```


>  ****Nunca**** versione o arquivo ``.env`` no Git. Ele já está incluído no ``.gitignore``.

API – Endpoints do Backend

Base URL: ``http://localhost:5000/api``

Autenticação — ``/api/auth``

Método	Rota	Descrição	Auth?
POST	<code>`/register`</code>	Cadastra um novo usuário	Não
POST	<code>`/login`</code>	Faz login e retorna JWT	Não

****POST /register**** — Corpo da requisição:

```
```json
{
 "name": "João Silva",
 "email": "joao@email.com",
 "password": "senha123"
}
```
```

****POST /login**** — Resposta de sucesso (``200``):

```
```json
{
 "token": "eyJhbGc...",
 "user": {
 "id": 1,
```

```
"name": "João Silva",
"email": "joao@email.com",
"role": "customer"
}
}
...
```

## Serviços — ``/api/services``

Método	Rota	Descrição	Auth?
GET	<code>`/`</code>	Lista todos os serviços disponíveis	Não
POST	<code>`/`</code>	Cria um novo serviço	Sim (Admin)

**\*\*GET /services\*\*** — Resposta de exemplo:

```
```json  
[  
  {  
    "id": 1,  
    "category": "Fade",  
    "name": "Mid/Low Fade",  
    "description": "Transição suave nas laterais...",  
    "price": 60.00,  
    "duration": 45  
  }  
]
```

]

Agendamentos — `/api/appointments`

Método	Rota	Descrição	Auth?
GET	/	Lista agendamentos do usuário (admin vê todos)	Sim (JWT)
POST	/	Cria um novo agendamento	Sim (JWT)

****POST /appointments**** — Corpo da requisição:

```
``json
```

```
{  
  "service_id": 1,  
  "appointment_date": "2025-08-15T10:00:00"  
}
```

```
``
```

> ⚠ O sistema verifica ****conflito de horários**** automaticamente. Caso o horário já esteja reservado, retorna `400`.

Produtos e Pedidos — `/api/products`

Método	Rota	Descrição	Auth?
GET	/	Lista todos os produtos	Não
POST	/orders	Cria um pedido	Sim (JWT)

POST /orders — Corpo da requisição:

```
``json
{
  "items": [{ "product_id": 1, "quantity": 2 }],
  "total_price": 99.90
}
``
---
```

Banco de Dados (In-Memory)

O projeto utiliza um ****banco de dados em memória**** (sem necessidade de MySQL, PostgreSQL ou outro banco externo). Os dados são armazenados em arrays JavaScript enquanto o servidor está em execução.

Estrutura das coleções:

...

inMemoryDB

- |— users[] — Usuários cadastrados
- |— services[] — Serviços (13 serviços pré-carregados)
- |— appointments[] — Agendamentos criados pelos clientes
- |— products[] — Produtos da loja
- |— orders[] — Pedidos realizados

...

Serviços pré-cadastrados (Seed):

Categoria	Serviços	
-----	-----	
Fade	Mid/Low Fade, Taper Fade, Faux Hawk Fade	
Clássicos	Buzz Cut, Crew Cut, Edgar (Takuash)	
Modernos	Pompadour Moderno, Mullet Moderno, Messy Hair	
Específicos	Cacheados & Crespos, Liso & Ondulado	
Barba	Barba Imperial, Combo Signature	

Usuário Administrador (pré-cadastrado):

Campo	Valor
Email	`adm@admin.com`
Senha	`123456`
Role	`admin`

> A senha é armazenada com hash `bcrypt` (salts = 10).

Frontend – Páginas e Componentes

-Rotas da Aplicação

Caminho	Componente	Descrição
`/`	`Home`	Página inicial com hero e serviços
`/login`	`Login`	Formulário de autenticação
`/register`	`Register`	Formulário de cadastro
`/scheduling`	`Scheduling`	Interface de agendamento
`/store`	`Store`	Loja de produtos
`/dashboard`	`Dashboard`	Painel administrativo

Descrição das Páginas

Home (/)

Página de apresentação com:

- **Hero Section** — Banner com imagem de fundo, nome da barbearia e CTAs para agendamento e loja.
- **Serviços em Destaque** — Cards com 3 serviços premium (Corte Executive, Ritual da Barba, Pacote Gold).
- **Diferenciais** — Seção com ícones (Elite, Confiança, Pontual, Premium).

Login (/login)

Formulário de login com campos `e-mail` e `senha`. Após autenticação bem-sucedida:

- Token JWT salvo no `localStorage`.
- Dados do usuário salvos no `localStorage`.
- Redirecionamento para `/scheduling`.

Register (/register)

Formulário de cadastro com `nome`, `e-mail` e `senha`. Após sucesso, redireciona para `/login`.

Scheduling (/scheduling)

Interface para agendamento de serviços:

- Lista de serviços disponíveis (consumidos da API).
- Seleção de data e horário.
- Requer usuário autenticado (JWT).

Store (`/store`)

Loja de produtos da barbearia, listando itens disponíveis para compra.

Dashboard (`/dashboard`)

Painel restrito para administradores com listagem de todos os agendamentos do sistema.

Componentes Reutilizáveis

- `Navbar.jsx` — Barra de navegação principal com links para todas as páginas. Exibe botões de Login/Logout condicionalmente com base no estado de autenticação.

- `Icons.jsx` — Biblioteca de ícones SVG customizados (Star, ShieldCheck, Clock, Award) utilizados em diversas páginas.

Autenticação e Segurança

Fluxo JWT

...

Cliente	Backend
---------	---------

--	--


```

|<-- { token } ---- |
|
|
|-- GET /appointments (Authorization: Bearer <token>) -->
|
|      |-- jwt.verify(token, JWT_SECRET)
|
|      |-- req.user = { id, role }
|
|<-- [ dados ] ---- |
...

```

Middleware de Autenticação

- **verifyToken** — Verifica se o token JWT é válido. Retorna ``403`` se não fornecido, ``401`` se inválido.

- **isAdmin** — Verifica se o usuário autenticado possui `role === 'admin'`. Retorna ``403`` para não-admins.

Armazenamento no Frontend

O token e dados do usuário são armazenados em `localStorage`:

```
````javascript
```

```
localStorage.setItem('token', token);
```

```
localStorage.setItem('user', JSON.stringify(user));
```

## ## Testes

Os testes são escritos com **Vitest** e **Testing Library**.

### ### Suítes de Teste

Arquivo	Cobertura	
-----	-----	
`Autenticacao.test.jsx`	Fluxos de login, cadastro e tokens	
`InterfaceDoUsuario.test.jsx`	Renderização de componentes e interações UI	
`ProcessosDeNegocio.test.jsx`	Regras de negócio (conflito de horários etc.)	

### Executar os Testes

```
``bash
```

#### Na pasta frontend:

```
npm test # Executa todos os testes uma vez
npm run test:ui # Abre a interface visual do Vitest
```

Scripts Disponíveis

Backend (`backend/`)

Comando	Ação
----- -----	
`npm start`	Inicia o servidor em modo produção (`node`)
`npm run dev`	Inicia com hot-reload via `nodemon`

Frontend (`frontend/`)

Comando	Ação
----- -----	
`npm run dev`	Servidor de desenvolvimento Vite (porta 5173)
`npm run build`	Gera a versão de produção em `/dist`
`npm run preview`	Pré-visualiza o build de produção
`npm test`	Executa os testes com Vitest
`npm run test:ui`	Abre UI visual dos testes
`npm run lint`	Verifica o código com ESLint

Fluxo da Aplicação

```mermaid

graph TD

```
A[Usuário acessa /] --> B{Autenticado?}
B -- Não --> C[Navega pela Home e Serviços]
C --> D[Clica em Reservar Agora]
D --> E[Redireciona para /login]
E --> F[Faz Login ou Cadastro em /register]
F --> G[Token JWT salvo no localStorage]
B -- Sim --> G
G --> H[Acessa /scheduling]
H --> I[Seleciona serviço e horário]
I --> J[POST /api/appointments com Bearer Token]
J --> K{Horário livre?}
K -- Sim --> L[Agendamento confirmado ✔]
K -- Não --> M[Erro: Horário já reservado ✖]
G --> N[Admin acessa /dashboard]
N --> O[Visualiza todos os agendamentos]
```

Considerações Finais

Banco de dados em memória: Os dados são ****resetados a cada reinício**** do servidor. Para persistência entre sessões, seria necessário integrar um banco de dados real como MySQL, PostgreSQL ou SQLite com arquivo.

Escalabilidade: A estrutura em camadas (routes → controllers → db) facilita a migração para um banco de dados real sem grandes alterações.

Segurança: Em produção, configure o CORS para aceitar apenas origens confiáveis e use variáveis de ambiente seguras para `JWT\_SECRET`.

Tudo de acordo com as normas aplicadas, métodos ágeis, tudo descrito no projeto e testes.

